Build a Plugin + Skill with your team

A practical ChatGPT workshop | About 2 hours | Work in pairs | Three sprints

Build one repeatable business workflow as a **skill**, place it in a **plugin**, connect only approved tools and sources, then test and improve it. Use invented or appropriately anonymised examples unless your organisation has explicitly approved the data and access arrangements.

The architecture

Plugin = the collection of related capabilities and shared instructions. **Skill** = one task-specific workflow and its checks. **Tools** = actions and integrations the plugin can use. **Reference files** = approved evidence, templates and examples. A skill does not automatically have access to your organisation's files: access must be configured and tested.

Example: Construction Operations plugin → PQQ Writer / Construction Programme Planner / Tender Reviewer skills → approved document access and templates.

Team members / plugin name / skill name:

_____

_____

Sprint 1 — Design and build (about 45 minutes)

1. Choose a task and decide where it belongs

Choose one recurring task: PQQ answers, tender compliance review, meeting minutes to actions, programme drafting, or weekly site reports.

This skill helps [role] produce [output] from [inputs].

_____

_____

Is this a new plugin or a new skill in an existing plugin? What other skills should remain separate?

Plugin and neighbouring skills:

_____

_____

2. Identify inputs, sources and permissions

Gather one test input, one or two approved reference examples, and a brief description of a good result. Identify where the source documents live and whether the plugin can actually read them.

Input / source	Where it lives	Access confirmed?
New task input		
Reference answers / template		
Feedback / house rules		

3. Interview the team

Start a chat with Plugin Creator or the plugin editing workflow available in your workspace. Ask for one question at a time.

Help me design a skill for [task] inside [plugin]. Interview me about users, inputs, approved sources, output format, tools, failure modes, human approvals and what must never happen. Ask one question at a time, then summarise the design. Do not invent organisational facts.

Sprint 1 — Build version one (continued)

4. Define the boundary between plugin and skill

Put shared identity, common restrictions and approved capabilities at plugin level. Put the exact task sequence, source-selection rules, output structure and task-specific checks in the skill. Do not duplicate rules unnecessarily.

Plugin-wide instructions	This skill's instructions
Shared organisational purpose	When this skill should trigger
Common confidentiality / approval rules	Task steps and decision points
Approved tools and access boundaries	Required output and source citations
	Task-specific exceptions and checks

5. Create or update the skill

Use Plugin Creator to add a skill to an existing plugin, or create a new plugin first. The skill should have a clear name, trigger description, ordered workflow, evidence rules, limits, output format and review checkpoints. If using SKILL.md, include a concise description of when to use it.

> Create a skill called [name] within the existing [plugin] plugin. Use the design summary. Specify when to use and not use it; exact steps; which approved sources and tools to consult; how to handle missing or conflicting evidence; required output; and human approval checkpoints. Show the proposed changes before applying them. Do not change other skills.

6. Set the guardrails

For every factual claim, figure, date or qualification, require a traceable source. Missing evidence must be marked as missing and referred to a person; never fabricated. For safety, quality and environmental tasks, the skill must not independently certify safety, compliance or suitability or replace competent professional assessment.

Worst plausible error and the rule that prevents it:

_____

_____

What must a human review or approve?

_____

_____

Approval boundary: drafting and internal analysis may be permitted; sending PQQs, submitting tenders, editing authoritative records or communicating externally must require explicit user approval.

7. Verify the source access

Run a small retrieval test. Ask the plugin to identify a specific fact from an approved reference and show its file/page/section. If access fails, stop and request the appropriate file or connection; do not pretend the source was read.

Access test and outcome:

_____

_____

Sprint 2 — Test, swap, improve (about 45 minutes)

8. Run a realistic example

Invoke the specific skill in the plugin and give it the test input. Check whether the correct skill ran, whether it used the right files, and whether it cited sources rather than guessing.

> Use the [skill] skill in [plugin] on this example. Produce the specified output, cite the approved evidence, identify missing facts and list all items needing human verification. Do not send or submit anything.

9. Swap with another pair

Another pair tests your skill using the same input and one tricky variation: a missing file, conflicting prior answers, an ambiguous question, or an attempted external submission. You test theirs.

10. Log gaps and make the smallest correction

Gap / failed test	Found by	Change to skill	Retested?

> The skill produced this problem: [gap]. Propose the smallest change to fix it. Show the exact before-and-after instruction, identify any effect on other skills, and wait for approval before updating. Then rerun the failed test.

Check for regression: does the new version still handle the original example and leave the other skills in the plugin behaving as intended?

What improved? What remains unresolved?

_____

_____

Sprint 3 — Package and govern (about 30 minutes)

11. Create the reference-source index

For each approved reference, record what it contains, its permitted use, owner and update date. State which source wins when two disagree; if this is unknown, require a human decision.

File / source	Use / do not use	Owner / updated

Conflict rule / source precedence:

_____

12. Package the plugin

Confirm the skill is present in the intended plugin, has a clear trigger description, can access its required tools and references, and does not overlap ambiguously with other skills. Keep an export or versioned copy of the instructions and source index according to your organisation's policy.

If plugin creation or editing is unavailable in your workspace, retain the skill instructions and test cases as a draft for later installation; a Project or prompt library is a fallback, not an equivalent plugin deployment.

13. Name an owner and a review date

Owner / next review date:

_____

Review when source documents, processes, permissions or platform capabilities change; otherwise schedule a regular review (for example, quarterly).

Final checklist

- ■ The plugin and skill have distinct responsibilities.
- ■ The skill has a clear trigger and non-trigger description.
- ■ Required files and tool permissions have been tested.
- ■ Claims and numbers trace back to approved sources.
- ■ Missing or conflicting information is surfaced, not invented.
- ■ External actions and submissions require explicit approval.
- ■ A second team has tested a failure case and the fix was retested.
- ■ The source index, owner, version and review date are recorded.

Change log

Date / version	What changed	Approved by

Workshop outcome: one tested skill, inside a clearly scoped plugin, with verified sources, explicit approval boundaries and an owner.